

Agent 智能体系统与 workflows

一、AI Agent 定义与核心公式

Agent 智能体系统与 workflows

一、AI Agent 定义与核心公式

AI Agent（人工智能代理）

是一个能够感知环境、进行决策并执行行动，以达成特定目标的智能软件实体。它不仅仅是回答问题的聊天机器

核心公式

Agent = LLM（大脑） + Planning（规划） + Tool use（执行）

更深层理解：为什么 Agent 是 AI 的下一个范式？

传统大模型（LLM）本质上是一个条件概率模型：给定上下文，预测下一个最可能的 token。它只能“说”，不能“做”。而 Agent 的核心突破在于：

1. 从被动应答到主动执行：LLM 只能在被提问时回答，Agent 能主动感知环境并采取行动
2. 从单轮推理到持续交互：LLM 是一次性生成，Agent 是观察 - 思考 - 行动的持续循环
3. 从纯文本空间到真实世界：LLM 的输出仅限于文本，Agent 能调用 API、执行代码、操作设备
4. 从无状态到有记忆：LLM 天然无状态，Agent 拥有记忆系统，能积累经验

用数学语言描述 Agent 的决策过程：

$$\pi(a_t | s_t, M) = \arg \max_{a_t} P(a_t | s_t; \theta, M)$$

其中 s_t 是当前状态（包括环境观察和记忆）， a_t 是行动， M 是记忆模块， θ 是模型的参数。Agent 在每一步选择最可能推动目标达成的行动。

...

二、AI Agent 结构组成

三大组成块

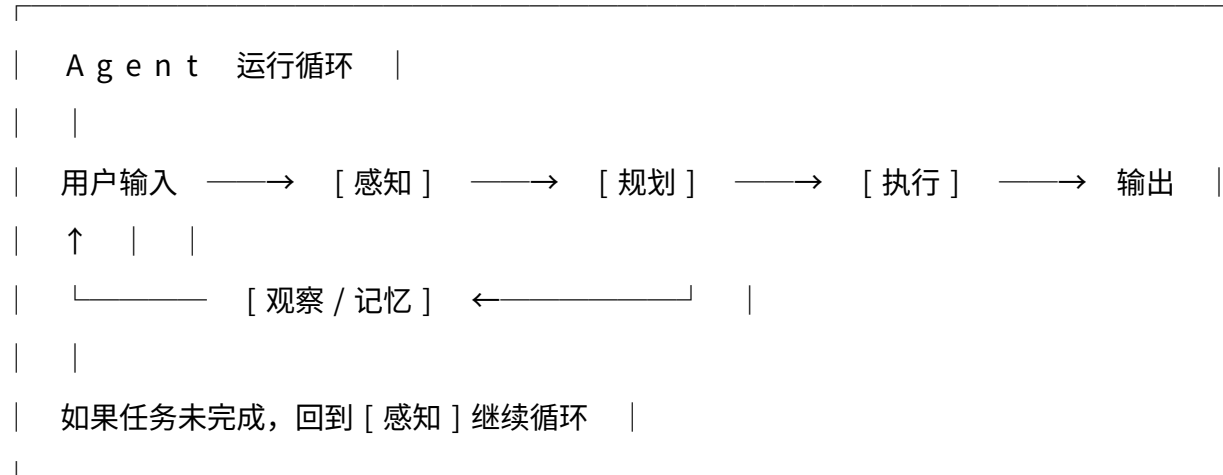
组成 说明 类比

目标 明确任务意图 员工收到任务书

逻辑 按规则拆成可执行步骤 员工制定工作计划

工具 通过代码或 A P I 让步骤落地 员工使用办公软件、打电话

完整运行流程



- 接收输入：理解用户的自然语言指令
- 判断当前任务：识别意图，提取关键信息
- 调用对应工具执行：选择合适的工具 / A P I
- 返回结果：获取工具执行结果
- 保留必要上下文：存入记忆系统
- 支持多轮连续操作：循环执行直到任务完成
- 遇阻时调整执行步骤：根据反馈重新规划

Agent 的五大核心能力

- 理解任务目标：明白你想要什么结果（语义理解）
 - 制定计划：思考如何达成目标（推理与规划）
 - 使用工具：调用各种资源和 A P I（工具调用）
 - 自我调整：根据反馈优化策略（反思与适应）
 - 持续执行：直到完成任务或遇到无法解决的问题（持久性）
- - -

三、与传统程序 / 普通大模型的差异

类型	特点	决策方式	灵活性
----	----	------	-----

传统程序 自动售货机模式：投币 → 按按钮 → 出商品（固定流程） 硬编码规则 无

普通大模型 只生成文本，不执行行动 统计推理 有限

AI Agent 私人助理模式：告诉需求 → 助理规划 → 完成任务并汇报 动态规划+工具调用 高

传统程序 vs Agent 的关键差异

传统程序：逻辑完全预定义

```
def traditionalweatherbot(userinput):
    if "天气" in userinput and "北京" in userinput:
        weather = callweatherapi("北京")
        return f"北京今天{weather['condition']}, {weather['temperature']}"
    elif "天气" in userinput and "上海" in userinput:
        weather = callweatherapi("上海")
        return f"上海今天{weather['condition']}, {weather['temperature']}"
    else:
        return "我只能查询北京和上海的天气" # 无法处理的输入直接失败
```

Agent：动态理解意图并选择工具

```
def agentweatherbot(userinput):
    # 1. LLM 理解意图：提取城市名、判断需要什么工具
    intent = llmunderstand(userinput) # 能处理任意城市、甚至模糊描述

    # 2. 动态选择和调用工具
    result = calltool(intent.toolname, intent.parameters)

    # 3. 根据结果继续推理
    if needsmoreinfo(result):
        result = callanothertool(...)

    return finalanswer

- - -
```

四、Agent 架构演进

4.1 三代架构

代次 架构 特点 代表 局限性

第一代 裸 L L M 直接对话，无工具 C h a t G P T 早期 只能生成文本，无法执行

第二代 L L M + W o r k f l o w 固定 workflow 编排 D i f f y、C o z e 流程固定，不够灵活

第三代 L L M + A g e n t 自主规划+工具调用 A u t o G P T、M a n u s 复杂任务可能出错，

架构演进的深层逻辑

第一代 → 第二代的驱动力：需要执行能力。纯 L L M 只能 " 说 "，无法调用 A P I、查询数据库、执

第二代 → 第三代的驱动力：需要灵活性。固定工作流（如 D i f f y 的 D A G）无法处理需要动态决策的复杂任务。A g e n t 能根据中间结果自主调整执行路径。

4 . 2 A g e n t 核心组件详解

组件 说明 实现方式 类比

L L M （大脑） 理解、推理、决策 G P T - 4、C l a u d e、D e e p S e e k 人的大脑皮层

P l a n n i n g （规划） 任务拆解与执行策略 R e A c t、P l a n - a n d - E x e c u t e、T r e

T o o l U s e （工具调用） 调用外部 A P I / 代码 F u n c t i o n C a l l i n g、M C P 人的

M e m o r y （记忆） 短期 / 长期记忆管理 向量数据库、对话历史 人的海马体

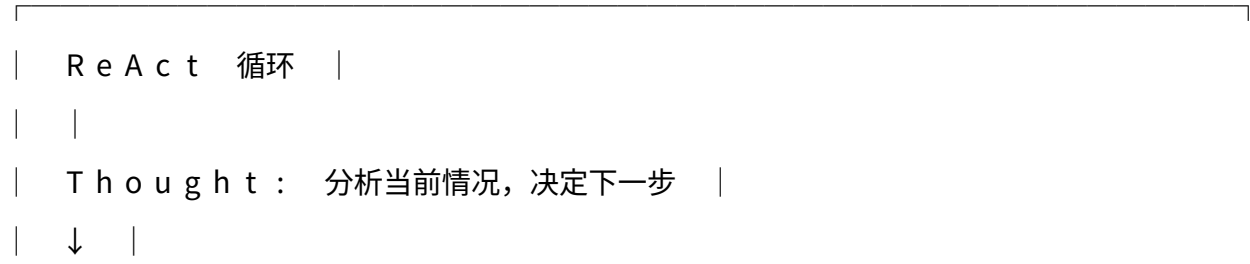
- - -

五、规划（P l a n n i n g）策略详解

5 . 1 R e A c t 模式——推理与行动的迭代循环

R e A c t （R e a s o n i n g + A c t i n g）是当前最主流的 A g e n t 规划模式，由 Y a c 提出。其核心思想是：推理指导行动，行动反馈推理，形成 T h o u g h t - A c t i o n - O b s e r v a

R e A c t 的基本流程



```

|   A c t i o n :   选择并执行一个工具   |
|   ↓   |
|   O b s e r v a t i o n :   获取工具执行结果   |
|   ↓   |
|   T h o u g h t :   基于观察继续推理...   |
|   ↓   |
|   (循环直到得出最终答案)   |

```

ReAct 的详细实现代码

```

import json
from openai import OpenAI

client = OpenAI(apikey="your-api-key")

=====

第一步：定义工具集合

=====

TOOLS = [
{
    "type": "function",
    "function": {
        "name": "searchweb",
        "description": "搜索互联网获取信息",
        "parameters": {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "搜索关键词"},
            },
            "required": ["query"]
        }
    }
},
{

```

```

" type": " function",
" function": {
" name": " getweather",
" description": " 获取指定城市的当前天气信息",
" parameters": {
" type": " object",
" properties": {
" city": { " type": " string", " description": " 城市名称" }
},
" required": [ " city" ]
}
},
{
" type": " function",
" function": {
" name": " calculate",
" description": " 执行数学计算",
" parameters": {
" type": " object",
" properties": {
" expression": { " type": " string", " description": " "
},
" required": [ " expression" ]
}
}
}
]

```

=====

第二步：工具执行器（实际执行工具调用）

=====

```

def executetool( toolname: str, arguments: dict ) -
" " " 根据工具名和参数执行工具，返回结果字符串 " " "

```

```

if toolname == "searchweb":
# 实际项目中替换为真实搜索API
return f"搜索结果：关于'{arguments['query']}'的最新信息..."
elif toolname == "getweather":
# 模拟天气API
mockdata = {
"北京": {"temp": 22, "condition": "晴朗", "humidity": 65},
"上海": {"temp": 28, "condition": "多云", "humidity": 75},
"深圳": {"temp": 31, "condition": "阵雨", "humidity": 85},
}
city = arguments["city"]
if city in mockdata:
w = mockdata[city]
return f"{city}当前天气：{w['condition']}，温度{w['temp']}°C，湿度{w['humidity']}%"
return f"未找到{city}的天气数据"
elif toolname == "calculate":
try:
result = eval(arguments["expression"]) # 注意：生产环境需限制表达式
return f"计算结果：{arguments['expression']} = {result}"
except Exception as e:
return f"计算错误：{e}"
else:
return f"未知工具：{toolname}"

```

=====

第三步：ReAct Agent 核心循环

=====

```

class ReActAgent:
def init(self, model="gpt-4", maxiterations=10):
self.model = model
self.maxiterations = maxiterations # 防止无限循环
self.conversationhistory = []

def run(self, userquery: str) -> str:

```

```
" " " 运行 ReAct 循环，直到得到最终答案或超过最大迭代次数 " " "
```

```
# 初始化对话
```

```
self.conversationhistory = [  
    {  
        "role": "system",  
        "content": (  
            "你是一个智能助手，能够通过思考和调用工具来回答问题。 \n"  
            "请按照 ReAct 模式工作： \n"  
            "1. Thought: 分析问题，思考下一步该做什么 \n"  
            "2. Action: 调用工具获取信息 \n"  
            "3. Observation: 观察工具返回的结果 \n"  
            "4. 重复以上步骤，直到能够给出最终答案 \n"  
            "当你得出最终答案时，请以 'Final Answer:' 开头。"  
        )  
    },  
    {"role": "user", "content": userquery}  
]
```

```
print(f"    用户问题: {userquery} \n")
```

```
for i in range(self.maxiterations):  
    print(f"- - - 第 {i+1} 轮迭代 - - -")
```

```
# 调用 LLM 获取下一步行动
```

```
response = client.chat.completions.create(  
    model=self.model,  
    messages=self.conversationhistory,  
    tools=TOOLS,  
    toolchoice="auto" # 让模型自主决定是否调用工具  
)
```

```
message = response.choices[0].message  
self.conversationhistory.append(message)
```



```

# 情况1：LLM 直接给出文本回复（可能包含最终答案）
if message.content:
    print(f"    Thought: {message.content[:200]}...")

# 检查是否已经得出最终答案
if "Final Answer:" in message.content or "最终答案" in message.content:
    print(f"\n    任务完成!")
    return message.content

# 情况2：LLM 请求调用工具
if message.toolcalls:
    for toolcall in message.toolcalls:
        functionname = toolcall.function.name
        functionargs = json.loads(toolcall.function.arguments)

    print(f"    Action: 调用工具 {functionname}({functionargs})")

# 执行工具
observation = executetool(functionname, functionargs)
print(f"    Observation: {observation[:200]}...")

# 将工具执行结果添加到对话历史
self.conversationhistory.append({
    "role": "tool",
    "toolcallid": toolcall.id,
    "content": observation
})
else:
# LLM 既没有调用工具，也没有给出最终答案
# 可能是纯推理步骤，继续下一轮
if not message.content or "Final Answer" not in message.content:
# 提示 LLM 继续思考
self.conversationhistory.append({
    "role": "user",
    "content": "请继续你的思考和行动，直到得出最终答案。"
})

```

```
} )
```

```
return "⚠ 达到最大迭代次数，任务未完成。"
```

```
=====
```

第四步：使用 ReAct Agent

```
=====
```

```
agent = ReActAgent()
```

```
result = agent.run("北京和上海今天哪个城市更适合户外活动？")
```

ReAct 的完整运行示例

用户问题：北京和上海今天哪个城市更适合户外活动？

- - - 第 1 轮迭代 - - -

Thought：需要比较北京和上海的天气来决定哪个更适合户外活动。先查北京天气。

Action：调用工具 `getweather({ "city": "北京" })`

Observation：北京当前天气：晴朗，温度 22℃，湿度 45%，3 级北风

- - - 第 2 轮迭代 - - -

Thought：北京天气不错。现在需要查上海天气来对比。

Action：调用工具 `getweather({ "city": "上海" })`

Observation：上海当前天气：多云，温度 28℃，湿度 72%，2 级东南风

- - - 第 3 轮迭代 - - -

Thought：现在两个城市天气都有了：

- 北京：晴朗、22℃、湿度 45%、3 级北风

- 上海：多云、28℃、湿度 72%、2 级东南风

从户外活动角度分析...

任务完成！

Final Answer：北京更适合户外活动。原因如下...

ReAct 的优缺点

优点 缺点

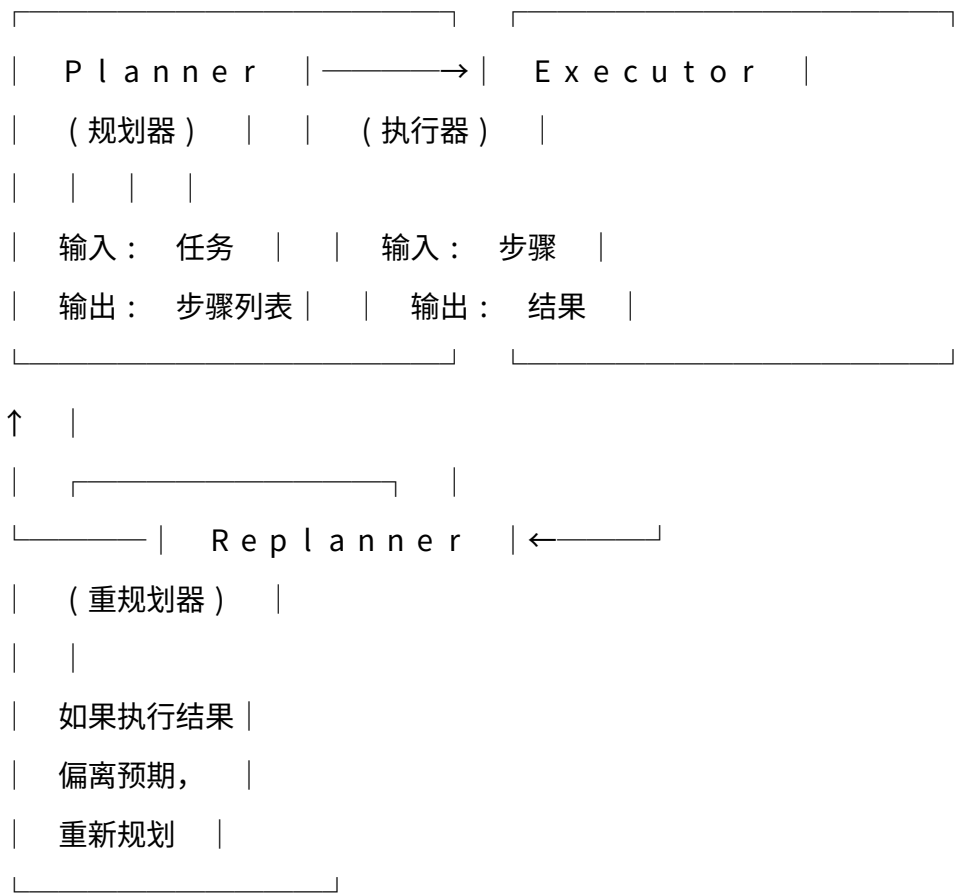
推理过程可解释、可追溯 Token 消耗大（每步都要完整上下文）

灵活，能处理意外情况 可能陷入循环（需要设最大迭代次数）
容易调试和改进 对 L L M 的指令跟随能力要求高
适合开放性任务 单步决策可能不够全局最优

5 . 2 P l a n - a n d - E x e c u t e 模式——先规划后执行

P l a n - a n d - E x e c u t e 将任务分解为两个阶段：先制定完整计划，再逐步执行。这比 R e A c t 更适合复杂的多步骤任务，因为 R e A c t 可能因为缺少全局视野而在某一步做出次优决策。

P l a n - a n d - E x e c u t e 的核心思想



P l a n - a n d - E x e c u t e 的详细实现

```
import json
from openai import OpenAI

client = OpenAI(apikey="your-api-key")

class PlanAndExecuteAgent:
    """Plan-and-Execute 模式的 Agent 实现"""
```

```

def init(self, model="gpt-4"):
    self.model = model

def plan(self, task: str) -> list:
    """规划阶段：将任务拆解为有序步骤列表"""
    response = client.chat.completions.create(
        model=self.model,
        messages=[
            {
                "role": "system",
                "content": (
                    "你是一个任务规划专家。将用户的任务拆解为具体的、可执行的步骤。\\n"
                    "每个步骤必须：\\n"
                    "1. 明确描述要做什么\\n"
                    "2. 依赖哪些前置步骤\\n"
                    "3. 预期输出是什么\\n\\n"
                    "以JSON列表格式输出，每个元素包含 step、description、dependson、"
                )
            },
            {
                "role": "user",
                "content": f"请规划以下任务：{task}"
            }
        ],
        response_format={"type": "jsonobject"}
    )

    result = json.loads(response.choices[0].message.content)
    return result.get("steps", [])

def executestep(self, step: dict, context: dict):
    """执行阶段：执行单个步骤"""
    response = client.chat.completions.create(
        model=self.model,
        messages=[
            {
                "role": "system",

```

```

"content": (
    "你是一个任务执行者。根据步骤描述和已有上下文，执行该步骤并返回结果。\\n"
    "如果需要调用外部工具，请使用 Function Calling.\\n"
    "返回执行结果的纯文本描述。"
)
},
{
    "role": "user",
    "content": (
        f"当前步骤：{step['description']}\\n"
        f"预期输出：{step.get('expectedoutput', '无')}\\n"
        f"已有上下文：{json.dumps(context, ensureascii=False, i
    )
    }
    ],
    tools=TOOLS # 复用前面定义的工具
)

message = response.choices[0].message

# 处理工具调用
if message.toolcalls:
    for toolcall in message.toolcalls:
        functionname = toolcall.function.name
        functionargs = json.loads(toolcall.function.arguments)
        result = executetool(functionname, functionargs)
        context[f"step{step.get('step', 'unknown')}toolresult"] = result

    return message.content or "执行完成"

def replan(self, originaltask: str, plan: list, e
    """重规划阶段：根据执行结果调整计划"""
    response = client.chat.completions.create(
        model=self.model,
        messages=[

```

```

{
    "role": "system",
    "content": (
        "你是一个任务规划专家。原始计划在执行中遇到了问题，请根据执行结果调整计划。 \n "
        "以 JSON 格式输出调整后的步骤列表。 "
    )
},
{
    "role": "user",
    "content": (
        f"原始任务：{originaltask} \n "
        f"原始计划：{json.dumps(plan, ensure_ascii=False)} \n "
        f"执行结果：{json.dumps(executedresults, ensure_ascii=False)} \n "
        f"请调整计划。 "
    )
},
],
responseformat={"type": "jsonobject"}
)

result = json.loads(response.choices[0].message.content)
return result.get("steps", [])

def run(self, task: str) -> str:
    """完整运行 Plan-and-Execute 流程"""
    print(f"    任务： {task} \n")

    # 阶段1：规划
    print("    规划阶段...")
    plan = self.plan(task)
    for i, step in enumerate(plan):
        print(f"    步骤{i+1}： {step['description']}")
    print()

    # 阶段2：逐步执行

```

```

print("    执行阶段...")
context = {}
for i, step in enumerate(plan):
    print(f"\n  执行步骤{i+1}: {step['description']}")
    result = self.executestep(step, context)
    context[f"step{i+1}result"] = result
    print(f"    结果: {result[:100]}...")

# 阶段3（可选）：检查是否需要重规划
# 这里简化处理，实际可以加入失败检测和重规划逻辑

return context

```

使用示例

```

agent = PlanAndExecuteAgent()
result = agent.run("帮我规划一个北京三日游，包括景点、交通和住宿建议")

```

Plan-and-Execute vs ReAct 的对比

维度 ReAct Plan-and-Execute

全局视野 每步局部决策 先有全局计划

适应性 高，随时调整 需要显式重规划

复杂任务 容易偏离目标 更稳定

简单任务 效率高 可能过度规划

适用场景 开放式探索 结构化任务

Token消耗 较高 可以更优化（仅发送当前步骤）

5.3 Tree of Thought (ToT) ——多路径推理

Tree of Thought 由 Yao et al. 2023

提出，核心思想是在推理的每一步探索多个可能的方向，选择最有前景的路径继续深入。这类似于国际象棋中的

ToT 的核心算法

问题

/ \

思路 A 思路 B

/ \ / \

步骤 A 1 步骤 A 2 步骤 B 1 步骤 B 2

↓ ↓ ↓ ↓

评估 : 8 评估 : 3 评估 : 6 评估 : 9

选择评估最高的路径继续

↓

步骤 B 2 → 步骤 B 2 a → 最终答案

T o T 的关键组件

- 1 . T h o u g h t G e n e r a t i o n (思路生成) : 在每一步生成 k 个候选思路
- 2 . S t a t e E v a l u a t i o n (状态评估) : 对每个思路进行评分 (0 - 1 0 分)
- 3 . S e a r c h A l g o r i t h m (搜索算法) : 选择最优路径继续 (B F S 或 D F S)
- 4 . B a c k t r a c k i n g (回溯) : 如果当前路径走到死胡同, 回退到之前的状态

T o T 的详细实现

```
import json
from openai import OpenAI
from typing import List, Dict, Optional

client = OpenAI(apikey="your-api-key")

class TreeOfThoughtAgent:
    """Tree of Thought 模式的 Agent 实现"""

    def __init__(self, model="gpt-4", numthoughts=3, maxdepth=None,
                 evaluationthreshold=None):
        self.model = model
        self.numthoughts = numthoughts # 每步生成的候选思路数
        self.maxdepth = maxdepth # 最大搜索深度
        self.evaluationthreshold = evaluationthreshold # 评估阈值

    def generatethoughts(self, problem: str, currentstate: str) -> List[str]:
        """生成多个候选思路"""
        response = client.chat.completions.create(
```



```

model = self.model ,
messages = [
{
"role": "system",
"content": (
"你是一个创造性问题解决者。给定问题和当前推理状态，"
f"生成 {num} 个不同的下一步推理思路。 \n "
"每个思路应该从不同角度推进问题的解决。 \n "
"以 JSON 列表格式输出，列表中的每个元素是一个思路字符串。 "
)
},
{
"role": "user",
"content":
f"问题：{problem} \n当前推理状态：{currentstate} \n \n请生成 {num}"
}
],
responseformat = { "type": "jsonobject" }
)

result = json.loads(response.choices[0].message.content)
return result.get("thoughts", []).[:num]

def evaluatethought(self, problem: str, thoughtchain: str):
"""评估一个思路的价值（0 - 10分）"""
response = client.chat.completions.create(
model=self.model,
messages = [
{
"role": "system",
"content": (
"评估给定的推理链对于解决问题的价值。 \n "
"评分标准： \n "
" - 10分：推理链已经解决问题 \n "

```

```

" - 7 - 9 分：推理链方向正确，接近解决 \ n "
" - 4 - 6 分：推理链有一定价值但方向可能不够好 \ n "
" - 0 - 3 分：推理链方向错误或无意义 \ n \ n "
" 只输出一个 0 到 10 的数字。 "
)
},
{
"role": "user",
"content": f"问题：{problem} \ n推理链：{thoughtchain}"
}
]
)

try:
score = float(response.choices[0].message.content)
return min(max(score, 0), 10) # 限制在 0 - 10 范围内
except:
return 5.0 # 默认中等分数

def run(self, problem: str) -> str:
"""运行 Tree of Thought 搜索"""
print(f"    问题： {problem} \ n")

# BFS 搜索
# 每个状态是一个 (推理链, 评分) 元组
currentstates = [("", 0)] # 初始状态为空推理链

bestsolution = None
bestscore = 0

for depth in range(self.maxdepth):
print(f"=== 搜索深度 {depth + 1} ===")
nextstates = []

for state, score in currentstates:

```

```

# 生成多个候选思路
thoughts = self.generatethoughts(problem, state)

for thought in thoughts:
# 构建新的推理链
newchain = f"{state}\n→ {thought}" if state else ""

# 评估新推理链
evaluation = self.evaluatethought(problem, newchain)
print(f" 评分 {evaluation:.1f}: {thought[:80]}...")

# 如果评估分数很高，可能是解决方案
if evaluation >= 9:
    print(f"\n  找到高分解决方案！")
    return newchain

# 只保留评估分数超过阈值的思路
if evaluation >= self.evaluationthreshold:
    nextstates.append((newchain, evaluation))

# 选择评分最高的几个状态继续搜索（剪枝）
nextstates.sort(key=lambda x: x[1], reverse=True)
currentstates = nextstates[:self.numthoughts] # 只保留numthoughts个

if not currentstates:
    print("所有思路均未通过评估阈值，搜索终止。")
    break

# 返回最佳推理链
if currentstates:
    bestchain, bestscore = max(currentstates, key=lambda x: x[1])
    return bestchain
return "未能找到解决方案。"

```

使用示例

```

totagent = TreeOfThoughtAgent(numthoughts=3, maxc

```

```
result = totagent.run("如何用最少的花费在一个陌生的城市度过充实的一周？")
```

5.4 Reflexion（反思机制）——从失败中学习

Reflexion 由 Shinn et al. 2023 提出，核心思想是：当 Agent 执行失败时，不是简单地重试，而是先反思失败原因，将反思结果存入记忆，在下次尝试时参考。

Reflexion 的工作流程



Reflexion 的详细实现

```
class ReflexionAgent:
    """Reflexion（反思）模式的 Agent 实现"""

    def __init__(self, model="gpt-4", max_attempts=3):
        self.model = model
        self.max_attempts = max_attempts
        self.reflections = [] # 存储反思记忆

    def act(self, task: str) -> tuple:
        """执行任务，返回（结果， 是否成功）"""
```

```
# 构建包含反思记忆的提示
```

```
reflectioncontext = ""
```

```
if self.reflections:
```

```
reflectioncontext = "\n\n之前的反思经验:\n"
```

```
for i, r in enumerate(self.reflections):
```

```
reflectioncontext += f" 第{i+1}次尝试的反思:{r}\n"
```

```
response = client.chat.completions.create(
```

```
model=self.model,
```

```
messages=[
```

```
{
```

```
"role": "system",
```

```
"content": (
```

```
"你是一个问题解决者。请尽力完成用户给出的任务。" +
```

```
reflectioncontext +
```

```
"\n\n如果之前有反思经验,请避免犯同样的错误。"
```

```
)
```

```
},
```

```
{"role": "user", "content": task}
```

```
],
```

```
tools=TOOLS
```

```
)
```

```
# 执行工具调用等操作...
```

```
message = response.choices[0].message
```

```
return message.content, True # 简化处理
```

```
def evaluate(self, task: str, result: str) -> tuple
```

```
"""评估执行结果,返回 (是否成功, 反馈)"""
```

```
response = client.chat.completions.create(
```

```
model=self.model,
```

```
messages=[
```

```
{
```

```
"role": "system",
```

```

"content": (
    "评估任务执行结果是否正确和完整。 \n"
    "返回 JSON 格式：{ \"success\": bool, \"feedback\": str"
)
},
{
    "role": "user",
    "content": f"任务：{task} \n执行结果：{result}"
},
response_format={ "type": "jsonobject" }
)

```

```

evalresult = json.loads(response.choices[0].message.content)
return evalresult.get("success", False), evalresult.get("feedback", "")

```

```

def reflect(self, task: str, result: str, feedback: str):
    """ 反思失败原因，生成反思记忆 """
    response = client.chat.completions.create(
        model=self.model,
        messages=[
            {
                "role": "system",
                "content": (
                    "你是一个自我反思专家。分析任务执行失败的原因，"
                    "给出具体的改进建议。 \n"
                    "反思应该包含： \n"
                    "1. 失败的具体原因 \n"
                    "2. 哪一步出了问题 \n"
                    "3. 下次应该如何避免 "
                )
            },
            {
                "role": "user",

```

```

"content": (
    f"任务：{task}\n"
    f"执行结果：{result}\n"
    f"评估反馈：{feedback}\n\n"
    "请反思失败原因。"
)
}
]
)

```

```

reflection = response.choices[0].message.content
self.reflections.append(reflection)
return reflection

```

```

def run(self, task: str) -> str:

```

```

    """运行 Reflexion 循环"""

```

```

    print(f"    任务： {task}\n")

```

```

    for attempt in range(1, self.maxattempts + 1):
        print(f"- - - 第 {attempt} 次尝试 - - - ")

```

```

    # 1. 执行任务

```

```

        result, = self.act(task)

```

```

        print(f"执行结果： {result[:100]}...")

```

```

    # 2. 评估结果

```

```

        success, feedback = self.evaluate(task, result)

```

```

        print(f"评估： {' 成功' if success else ' 失败' } - {feedback}")

```

```

        if success:

```

```

            print(f"\n    任务在第 {attempt} 次尝试中完成!")

```

```

            return result

```

```

    # 3. 反思失败原因

```

```

        reflection = self.reflect(task, result, feedback)

```

```

        print(f"反思： {reflection[:100]}...\n")

```

```
print(f"⚠ 达到最大尝试次数 {self.max_attempts}，任务未完成。")
return result
```

使用示例

```
reflexionagent = ReflexionAgent(max_attempts=3)
result =
reflexionagent.run("编写一个Python函数，将CSV文件转换为JSON格式，
```

5.5 规划策略选择指南

任务特征 推荐策略 原因

简单查询（查天气、翻译） ReAct 单步即可完成，无需全局规划

多步骤线性任务 Plan-and-Execute 需要全局规划确保步骤完整

创造性 / 开放式问题 Tree of Thought 需要探索多种可能性

容易出错的编程任务 Reflexion 需要从失败中学习和改进

复杂混合任务 ReAct + Reflexion 结合灵活性和自我纠错能力

- - -

六、工具调用（Tool Use）详解

6.1 Function Calling 完整实现

Function Calling 是 LLM 与外部世界交互的核心机制。其工作原理是：LLM 不直接执行工具，而是输出结构化的工具调用请求（函数名 + 参数），由外部框架负责执行并将结果返回给 LLM。

OpenAI Function Calling 的详细 API 使用

```
from openai import OpenAI
import json
```

```
client = OpenAI(api_key="your-api-key")
```

```
=====
```

步骤1：定义工具（Tools）

```
=====
```


工具定义遵循 JSON Schema 规范

```
tools = [
{
  "type": "function",
  "function": {
    "name": "getstockprice",
    "description": "获取指定股票的当前价格和涨跌信息",
    "parameters": {
      "type": "object",
      "properties": {
        "symbol": {
          "type": "string",
          "description": "股票代码, 如 AAPL、TSLA、600519 (贵州茅台)"
        },
        "market": {
          "type": "string",
          "enum": [ "US", "HK", "CN" ],
          "description": "市场: 美股、港股、A股"
        }
      }
    },
    "required": [ "symbol" ],
    "additionalProperties": False # 不允许额外参数
  }
},
{
  "type": "function",
  "function": {
    "name": "calculateportfolio",
    "description": "计算投资组合的总市值和收益率",
    "parameters": {
      "type": "object",
      "properties": {
```

```

"holdings": {
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "symbol": {"type": "string"},
      "shares": {"type": "number"},
      "costbasis": {"type": "number"}
    },
    "required": ["symbol", "shares", "costbasis"]
  },
  "description": "持仓列表"
},
"required": ["holdings"],
"additionalProperties": False
}
}
]

```

=====

步骤2：实现工具执行函数

=====

```

def getstockprice(symbol: str, market: str = "US"):
    """ 获取股票价格（模拟实现） """
    mockdata = {
        "AAPL": {"price": 178.50, "change": +2.30, "change_p": 1.3},
        "TSLA": {"price": 245.20, "change": -5.10, "change_p": -2.1},
        "600519": {"price": 1688.00, "change": +12.50, "change_p": 0.7}
    }
    if symbol in mockdata:
        data = mockdata[symbol]
        return {"symbol": symbol, "market": market, "data": data}
    else:
        return {"symbol": symbol, "market": market, "data": None}

```

```

return { "error": f"未找到股票 {symbol}" }

def calculateportfolio(holdings: list) -> dict:
    """ 计算投资组合 """
    totalvalue = 0
    totalcost = 0
    for h in holdings:
        # 先获取当前价格
        pricedata = getstockprice(h["symbol"])
        if "error" not in pricedata:
            currentvalue = pricedata["price"] * h["shares"]
            costvalue = h["costbasis"] * h["shares"]
            totalvalue += currentvalue
            totalcost += costvalue

    return {
        "totalvalue": round(totalvalue, 2),
        "totalcost": round(totalcost, 2),
        "profitloss": round(totalvalue - totalcost, 2),
        "returnpct": round((totalvalue - totalcost) / totalcost, 2)
    }

```

工具名到执行函数的映射

```

TOOLFUNCTIONS = {
    "getstockprice": getstockprice,
    "calculateportfolio": calculateportfolio,
}

```

=====

步骤3：处理多轮工具调用

=====

```

def runconversationwithtools(usermessage: str, messages: list):
    """ 处理包含多轮工具调用的完整对话 """

```

```

messages = [

```

```

{
    "role": "system",
    "content": "你是一个投资顾问助手，可以查询股票价格和计算投资组合。"
},
{"role": "user", "content": usermessage}
]

for roundnum in range(maxrounds):
    print(f"\n - - 对话轮次 {roundnum + 1} - - ")

# 调用 LLM
response = client.chat.completions.create(
    model="gpt-4",
    messages=messages,
    tools=tools,
    toolchoice="auto" # auto: LLM自主决定是否调用工具
# 其他选项：
# toolchoice="none": 禁止调用工具
# toolchoice={"type": "function", "function": {"name": "get_stock_price"}}
)

assistantmessage = response.choices[0].message
messages.append(assistantmessage)

# 检查是否有工具调用
if assistantmessage.toolcalls:
    for toolcall in assistantmessage.toolcalls:
        functionname = toolcall.function.name
        functionargs = json.loads(toolcall.function.arguments)

    print(f"        调用: {functionname}({json.dumps(functionargs)})"

# 执行工具
try:
    result = TOOLFUNCTIONS[functionname](functionargs)

```

```

resultstr = json.dumps(result, ensureascii=False)
except Exception as e:
    resultstr = json.dumps({"error": str(e)})
    print(f" ⚠️ 工具执行错误: {e}")

    print(f"      结果: {resultstr[:100]}...")

# 将结果添加到对话
messages.append({
    "role": "tool",
    "toolcallid": toolcall.id,
    "content": resultstr
})
else:
    # LLM 没有调用工具, 说明已经准备好回答
    print(f"\n      最终回复: {assistantmessage.content}")
    return assistantmessage.content

return "达到最大轮次限制。"

```

=====

步骤4：错误处理和重试机制

=====

```

class RobustToolCaller:
    """带错误处理和重试机制的工具调用器"""

    def __init__(self, maxretries=3, retrydelay=1.0):
        self.maxretries = maxretries
        self.retrydelay = retrydelay

    def executewithretry(self, func, kwargs):
        """带重试的工具执行"""
        import time

        lasterror = None
        for attempt in range(self.maxretries):

```

```

try:
    result = func(kwargs)

# 检查工具返回的业务错误
if isinstance(result, dict) and "error" in result:
    raise ValueError(f"业务错误: {result['error']}")

return result

except Exception as e:
    last_error = e
    print(f" ⚠ 第 {attempt + 1} 次执行失败: {e}")
    if attempt < self.max_retries - 1:
        time.sleep(self.retry_delay (attempt + 1)) # 指数退避

    raise Exception(f"工具执行失败, 已重试 {self.max_retries} 次。")

def validate_tool_output(self, toolname: str, output):
    """ 验证工具输出的合法性 """
    if toolname == "getstockprice":
        return isinstance(output, dict) and "price" in output
    elif toolname == "calculateportfolio":
        return isinstance(output, dict) and "totalvalue" in output
    return True

def safe_execute(self, toolname: str, arguments: dict):
    """ 安全执行工具调用, 包含验证和重试 """
    if toolname not in TOOLFUNCTIONS:
        return json.dumps({"error": f"未知工具: {toolname}"})

    try:
        result = self.executewithretry(TOOLFUNCTIONS[toolname], arguments)

        if not self.validate_tool_output(toolname, result):
            return json.dumps({"error": "工具输出验证失败", "output": result})

        return json.dumps(result, ensure_ascii=False)

```

```
except Exception as e:
    return json.dumps( {"error": str(e)} )
```

使用示例

```
caller = RobustToolCaller(max_retries=3)
result = run_conversation_with_tools(
    "我持有100股AAPL（成本150）和50股TSLA（成本200），帮我看看我的投资组合表现如何",
    caller,
    tools,
    max_turns=10,
    max_tokens=1024,
    temperature=0.5,
    seed=42,
)
```

Function Calling 的常见误区

误区 正确理解

LLM 会执行工具 LLM 只生成工具调用请求，执行由外部代码完成

工具参数总是正确的 需要验证参数类型和范围, L L M 可能生成非法参数

一次只能调用一个工具 OpenAI 支持并行调用多个工具 (parallel tool calls)

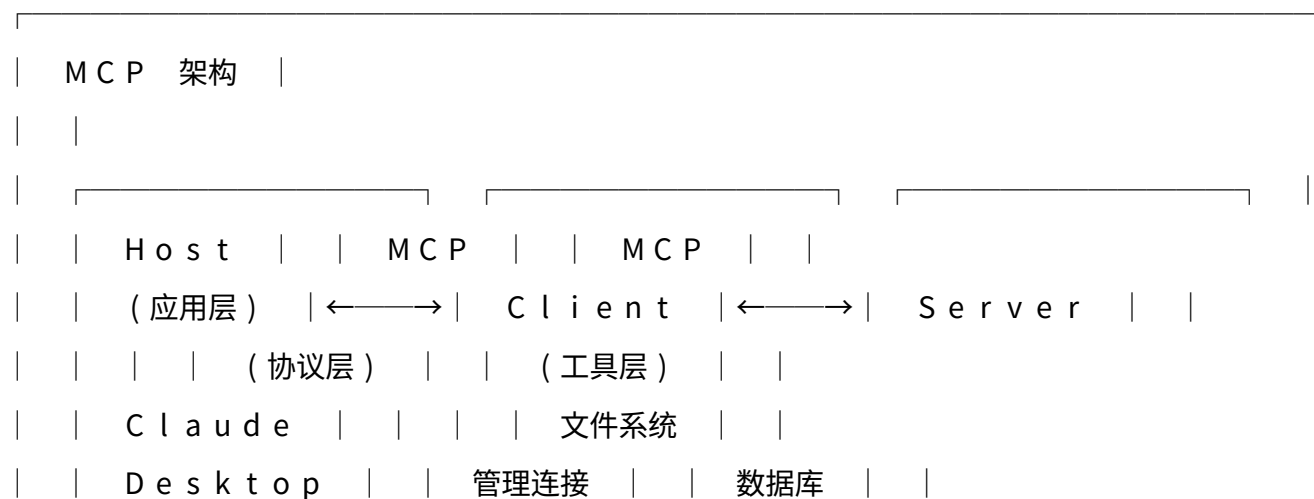
工具描述不重要 工具描述是 LLM 理解工具的唯一依据，必须精确

`toolchoice=auto` 总是调用工具 `auto` 模式下 LLM 可能选择不调用工具而直接回

6.2 MCP (Model Context Protocol) 详解

MCP 是 Anthropic 于 2024 年提出的开放协议，旨在标准化 LLM 与外部工具和数据源的交互方式。类比：如果 Function Calling 是 "点对点连接"，MCP 是 "统一接口"。

MC P 的架构设计



		I D E			处理协议			W e b 搜索					
											G i t H u b		

核心概念：

- H o s t ：发起连接的应用（如 C l a u d e D e s k t o p 、 C u r s o r I D E ）
- C l i e n t ：与 S e r v e r 保持 1 : 1 连接的协议客户端
- S e r v e r ：提供具体工具和资源的服务端

M C P 的三种核心能力

能力 说明 示例

T o o l s L L M 可调用的函数 搜索文件、查询数据库、执行代码

R e s o u r c e s L L M 可读取的数据 文件内容、数据库记录、A P I 响应

P r o m p t s 预定义的提示模板 代码审查模板、文档生成模板

M C P S e r v e r 的实现示例

使用 m c p 包创建一个简单的 M C P S e r v e r

```
pip install mcp
```

```
from mcp.server import Server
from mcp.types import Tool, TextContent
import asyncio
```

创建 M C P S e r v e r 实例

```
server = Server("my-tools-server")
```

```
=====
```

定义工具

```
=====
```

```
@server.list_tools()
```

```
async def list_tools() -> list[Tool]:
```

```
""" 声明服务器提供的所有工具 """
```

```
return [
```



```
Tool (
name = "readfile",
description = "读取指定路径的文件内容",
inputSchema = {
"type": "object",
"properties": {
"path": {
"type": "string",
"description": "文件的绝对路径"
}
},
"required": ["path"]
},
Tool (
name = "searchcode",
description = "在代码库中搜索匹配的代码",
inputSchema = {
"type": "object",
"properties": {
"query": {
"type": "string",
"description": "搜索关键词或正则表达式"
},
"filepattern": {
"type": "string",
"description": "文件匹配模式, 如 '.py'"
}
},
"required": ["query"]
},
Tool (
```

```

name = "runpython",
description = "执行Python代码并返回输出",
inputSchema = {
    "type": "object",
    "properties": {
        "code": {
            "type": "string",
            "description": "要执行的Python代码"
        }
    },
    "required": ["code"]
}
)
]

```

```

@server.calltool()
async def calltool(name: str, arguments: dict) -> list[TextContent]:
    """处理工具调用请求"""
    if name == "readfile":
        path = arguments["path"]
        try:
            with open(path, "r", encoding="utf-8") as f:
                content = f.read()
            return [TextContent(type="text", text=content)]
        except FileNotFoundError:
            return [TextContent(type="text", text=f"错误:文件 {path} 不存在")]
        except Exception as e:
            return [TextContent(type="text", text=f"错误:{str(e)}")]

    elif name == "searchcode":
        # 实现搜索逻辑...
        return [TextContent(type="text", text="搜索结果...")]

    elif name == "runpython":

```

```

# 安全地执行Python代码（沙盒环境）
code = arguments["code"]
try:
# 注意：生产环境需要更安全的执行方式
localvars = {}
exec(code, {"builtins": {}}, localvars)
return [TextContent(type="text", text=str(localvars))]
except Exception as e:
return [TextContent(type="text", text=f"执行错误：{str(e)}")]

return [TextContent(type="text", text=f"未知工具：{name}")]

```

启动服务器

```

async def main():
from mcp.server.stdio import stdio_server

async with stdio_server() as (readstream, writestream):
await server.run(
readstream,
writestream,
server.create_initialization_options()
)

if __name__ == "__main__":
asyncio.run(main())

```

常用 MCP 工具一览

工具名 功能 安装方式

@modelcontextprotocol/server-filesystem	文件系统读写	npm install -g @modelcontextprotocol/server-filesystem
@modelcontextprotocol/server-github	GitHub 操作	npm install -g @modelcontextprotocol/server-github
@modelcontextprotocol/server-postgres	PostgreSQL 数据库	npm install -g @modelcontextprotocol/server-postgres
@modelcontextprotocol/server-brave-search	Web 搜索	npm install -g @modelcontextprotocol/server-brave-search
@modelcontextprotocol/server-puppeteer	浏览器自动化	npm install -g @modelcontextprotocol/server-puppeteer
@modelcontextprotocol/server-sqlite	SQLite 数据库	npm install -g @modelcontextprotocol/server-sqlite

- - -

七、记忆系统（Memory）详解

记忆系统是 Agent 区别于无状态 LLM 调用的关键组件。人类依靠记忆积累经验、维持连贯对话，同样需要记忆来保持上下文和积累知识。

7.1 记忆类型详解

类型 说明 持续时间 实现方式 类比

短期记忆 当前对话上下文 单次会话 对话历史窗口 人的工作记忆

长期记忆 跨会话的持久知识 永久 向量数据库、知识图谱 人的长期记忆

工作记忆 当前任务的中间状态 任务执行期间 Scratchpad 人的草稿纸

7.2 短期记忆的实现

对话窗口管理

```
class ConversationWindowManager:
    """ 对话窗口管理器 - 控制发送给 LLM 的上下文长度 """

    def __init__(self, max_tokens=4096, model="gpt-4"):
        self.max_tokens = max_tokens
        self.model = model
        self.messages = []

    def estimate_tokens(self, text: str) -> int:
        """ 估算文本的 token 数（粗略：中文约 1 字 = 2 token，英文约 4 字符 = 1 token） """
        # 实际应用中 使用 tiktoken 库精确计算
        return len(text) // 2 # 简化估算

    def add_message(self, role: str, content: str):
        """ 添加消息并检查窗口限制 """
        self.messages.append({"role": role, "content": content})
        self.trim_if_needed()
```

```

def trim_if_needed(self):
    """ 如果超出窗口限制，移除最早的消息（保留system消息） """
    total_tokens = sum(self.estimate_tokens(m["content"]) for m in self.messages)

    while total_tokens > self.max_tokens and len(self.messages) > 1:
        # 找到第一条非system消息并移除
        for i, msg in enumerate(self.messages):
            if msg["role"] != "system":
                removed = self.messages.pop(i)
                total_tokens -= self.estimate_tokens(removed["content"])
                break

def get_messages(self) -> list:
    """ 获取当前窗口内的消息 """
    return self.messages.copy()

```

摘要压缩

```

class ConversationSummarizer:
    """ 对话摘要压缩 - 用LLM总结历史对话，释放窗口空间 """

    def __init__(self, client, model="gpt-4", summary_trigger_tokens=1000):
        self.client = client
        self.model = model
        self.summary_trigger = summary_trigger_tokens # 触发摘要的tokens
        self.summary = "" # 存储摘要

    def should_summarize(self, messages: list) -> bool:
        """ 判断是否需要摘要压缩 """
        total_tokens = sum(len(m["content"]) for m in messages) // 2
        return total_tokens > self.summary_trigger

    def summarize(self, messages: list) -> str:
        """ 生成对话摘要 """
        # 提取需要摘要的消息（排除system消息）
        conversation_text = ""

```

```

for msg in messages:
    if msg["role"] != "system":
        conversationtext += f"{msg['role']}: {msg['content']}

response = self.client.chat.completions.create(
model=self.model,
messages=[
{
"role": "system",
"content": (
"请将以下对话历史压缩为简洁的摘要，保留所有关键信息：\n"
"- 讨论的主要话题\n"
"- 做出的重要决定\n"
"- 提到的关键事实和数据\n"
"- 未解决的问题\n"
)
},
{"role": "user", "content": conversationtext}
]
)

self.summary = response.choices[0].message.content
return self.summary

def getcompressedmessages(self, messages: list) -
""" 获取压缩后的消息列表 """
if not self.shouldsummarize(messages):
    return messages

# 生成摘要
self.summarize(messages)

# 构建压缩后的消息：system + 摘要 + 最近几轮对话
compressed = [m for m in messages if m["role"] !=
compressed.append({

```

```
"role": "system",
"content": f"[对话历史摘要]\n{self.summary}"
})
```

保留最近3轮对话

```
recent = [m for m in messages if m["role"] != "system"]
compressed.extend(recent)

return compressed
```

7.3 长期记忆的实现

基于向量数据库的长期记忆

```
import numpy as np
from dataclasses import dataclass, field
from typing import List, Optional

@dataclass
class MemoryItem:
    """单条记忆项"""
    content: str # 记忆内容
    embedding: List[float] # 向量表示
    metadata: dict = field(default_factory=dict) # 元数据
    importance: float = 1.0 # 重要性评分

class VectorMemoryStore:
    """基于向量数据库的长期记忆存储"""

    def __init__(self, embeddingdim=1536):
        self.embeddingdim = embeddingdim
        self.memories: List[MemoryItem] = []

    def get_embedding(self, text: str) -> List[float]:
        """获取文本的向量表示（使用OpenAI Embedding API）"""
        # 实际实现中调用 embedding API
        # response = client.embeddings.create(input=text,
```

```

# return response.data[0].embedding
return np.random.randn(self.embeddingdim).tolist()

def addmemory(self, content: str, metadata: dict):
    """ 添加一条记忆 """
    embedding = self.getembedding(content)
    memory = MemoryItem(
        content=content,
        embedding=embedding,
        metadata=metadata or {},
        importance=importance
    )
    self.memories.append(memory)

def cosinesimilarity(self, vec1: List[float], vec2: List[float]):
    """ 计算余弦相似度 """
    a = np.array(vec1)
    b = np.array(vec2)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

def retrieve(self, query: str, topk: int = 5, min_similarity: float = 0.5) -> List[MemoryItem]:
    """ 检索与查询最相关的记忆 """
    queryembedding = self.getembedding(query)

    # 计算每条记忆与查询的相似度
    scoredmemories = []
    for memory in self.memories:
        similarity = self.cosinesimilarity(queryembedding, memory.embedding)
        # 综合考虑相似度和重要性
        score = similarity * memory.importance
        scoredmemories.append((memory, score, similarity))

    # 按分数排序, 取 top - k
    scoredmemories.sort(key=lambda x: x[1], reverse=True)

```



```

# 过滤低相似度的结果
results = [
    (mem, sim) for mem, sim in scoredmemories
    if sim >= minsimilarity
][:topk]

return results

def decaymemories(self, decayfactor: float = 0.99)
    """ 记忆衰减 - 模拟人类遗忘曲线 """
    for memory in self.memories:
        memory.importance = decayfactor

# 移除重要性过低的记忆
self.memories = [m for m in self.memories if m.in

```

使用示例

```
memorystore = VectorMemoryStore()
```

存储记忆

```

memorystore.addmemory("用户喜欢Python编程语言", metadata={
    'importance': 0.8})
memorystore.addmemory("用户的项目使用React和TypeScript",
    importance=0.9)
memorystore.addmemory("用户之前遇到过React Hooks的问题", me
    importance=0.7)

```

检索相关记忆

```

results = memorystore.retrieve("用户用什么前端框架?", topk=5)
for memory, similarity in results:
    print(f"相似度{similarity:.2f}: {memory.content}")

```

基于知识图谱的长期记忆

```

class KnowledgeGraphMemory:
    """ 基于知识图谱的长期记忆 """

```

```

def init(self):
    self.entities = {} # 实体: {name: {type, properties}}
    self.relations = [] # 关系: [(subject, predicate, object, properties)]

def addentity(self, name: str, entitytype: str, properties: dict):
    """添加实体"""
    self.entities[name] = {
        "type": entitytype,
        "properties": properties or {}
    }

def addrelation(self, subject: str, predicate: str, object: str, properties: dict):
    """添加关系"""
    self.relations.append((subject, predicate, object, properties))

def queryentity(self, name: str) -> dict:
    """查询实体及其所有关系"""
    if name not in self.entities:
        return {"error": f"实体 {name} 不存在"}

    entity = self.entities[name]
    related = []

    # 查找与该实体相关的所有关系
    for s, p, o, props in self.relations:
        if s == name:
            related.append({"direction": "outgoing", "predicate": p, "object": o, "properties": props})
        elif o == name:
            related.append({"direction": "incoming", "predicate": p, "subject": s, "properties": props})

    return {"entity": entity, "relations": related}

def querypath(self, start: str, end: str, maxdepth: int):
    """查询两个实体之间的路径(BFS)"""
    from collections import deque

```

```

queue = deque([(start, [start])])
visited = {start}

while queue:
    current, path = queue.popleft()

    if current == end:
        return path

    if len(path) > maxdepth:
        continue

    # 查找相邻实体
    for s, p, o, in self.relations:
        neighbor = None
        if s == current and o not in visited:
            neighbor = o
        elif o == current and s not in visited:
            neighbor = s

        if neighbor:
            visited.add(neighbor)
            queue.append((neighbor, path + [neighbor]))

    return []

```

使用示例

```

kg = KnowledgeGraphMemory()
kg.addentity("张三", "Person", {"role": "developer"})
kg.addentity("React项目", "Project", {"stack": "React"})
kg.addentity("Python", "Technology", {"type": "language"})
kg.addrelation("张三", "workson", "React项目")
kg.addrelation("张三", "prefers", "Python")

print(kg.queryentity("张三"))

```

7.4 工作记忆 (Scratchpad 机制)

```

class Scratchpad:
    """工作记忆 - 存储当前任务的中间状态"""

    def __init__(self):
        self.state = {} # 当前状态
        self.history = [] # 操作历史
        self.variables = {} # 中间变量

    def set(self, key: str, value):
        """设置状态变量"""
        oldvalue = self.state.get(key)
        self.state[key] = value
        self.history.append({
            "action": "set",
            "key": key,
            "oldvalue": oldvalue,
            "newvalue": value
        })

    def get(self, key: str, default=None):
        """获取状态变量"""
        return self.state.get(key, default)

    def getcontextstring(self) -> str:
        """将工作记忆格式化为可注入提示的字符串"""
        lines = ["[当前工作记忆状态]"]
        for key, value in self.state.items():
            lines.append(f" {key}: {value}")
        return "\n".join(lines)

    def clear(self):
        """清空工作记忆（新任务开始时）"""
        self.state.clear()
        self.history.clear()

```

```
self.variables.clear()
```

7.5 记忆检索策略

```
class HybridMemoryRetriever:
```

```
    """混合记忆检索策略"""
```

```
    def init(self, shortterm, longterm, working):
```

```
        self.shortterm = shortterm # 短期记忆管理器
```

```
        self.longterm = longterm # 长期记忆存储
```

```
        self.working = working # 工作记忆
```

```
    def retrieveforquery(self, query: str, topk: int
```

```
    """为当前查询检索所有相关记忆"""
```

```
        contextparts = []
```

```
        # 1. 工作记忆（最高优先级）
```

```
        workingcontext = self.working.getcontextstring()
```

```
        if workingcontext:
```

```
            contextparts.append(f"【工作记忆】\n{workingcontext}")
```

```
        # 2. 短期记忆（最近对话）
```

```
        recentmessages = self.shortterm.getmessages()[-6:]
```

```
        if recentmessages:
```

```
            shorttermtext = "\n".join(
```

```
                f" {m['role']}: {m['content'][:100]}..."
```

```
                for m in recentmessages
```

```
            )
```

```
            contextparts.append(f"【近期对话】\n{shorttermtext}")
```

```
        # 3. 长期记忆（相关历史）
```

```
        longtermresults = self.longterm.retrieve(query, t
```

```
        if longtermresults:
```

```
            longtermtext = "\n".join(
```

```
                f" [{sim:.2f}] {mem.content}"
```

```
                for mem, sim in longtermresults
```

```
)
contextparts.append(f"【相关记忆】\n{longtermtext}")

return "\n\n".join(contextparts)

- - -
```

八、完整代码示例：天气穿衣助手 Agent

```
class WeatherAgent:
    """一个完整的 ReAct 模式 Agent 示例"""

    def __init__(self):
        self.memory = [] # 简单的记忆存储
        self.tools = {
            'getweather': self.getweatherapi,
            'giveadvice': self.generateadvice
        }

    # 工具1： 调用天气API
    def getweatherapi(self, city):
        print(f"[Agent 行动] 正在查询{city}的天气...")
        mockdata = {'city': city, 'temp': 22, 'condition': '晴天'}
        return mockdata

    # 工具2： 根据天气生成建议
    def generateadvice(self, weatherdata):
        temp = weatherdata['temp']
        condition = weatherdata['condition']
        advice = f"当前{weatherdata['city']}气温{temp}°C，天气{condition}"
        if temp > 25:
            advice += "建议穿短袖、短裤。"
        elif temp > 15:
            advice += "建议穿长袖T恤、薄外套。"
        else:
            advice += "建议穿毛衣、厚外套。"
```

```
return advice
```

```
# 规划与执行核心
```

```
def run(self, userinput):  
    print(f"[用户指令] {userinput}")
```

```
# 步骤1： 规划 - 从指令中提取关键信息
```

```
if "天气" in userinput and "北京" in userinput:  
    city = "北京"  
else:  
    return "请告诉我您想查询哪个城市的天气？"
```

```
# 步骤2： 行动 - 调用工具获取天气
```

```
weatherinfo = self.tools.getweather'  
self.memory.append({'step': 'fetchdweather', 'data': weatherinfo})
```

```
# 步骤3： 行动 - 调用工具生成建议
```

```
finaladvice = self.tools.giveadvice'  
self.memory.append({'step': 'generatedadvice', 'data': finaladvice})  
  
return finaladvice
```

使用Agent

```
agent = WeatherAgent()  
result = agent.run("我想知道北京的天气，该怎么穿衣服？")  
  
---
```

九、多智能体系统（Multi-Agent）详解

9.1 协作模式

模式	说明	示例	适用场景
串行	依次执行，前一个输出是后一个输入	研究员→撰写者→审核者	内容生产
并行	多个Agent同时执行不同子任务	多人同时搜索不同主题	信息收集
层级	管理者分配任务给执行者	项目经理分配任务	复杂项目管理

辩论 多Agent 辩论达成共识 正方vs反方辩论 决策优化

混合 组合多种模式 层级管理+并行执行 大型项目

9.2 CrewAI 详细使用教程

CrewAI 是一个角色驱动的多Agent框架，核心理念是：定义角色 → 组建团队 → 分配任务

```
pip install crewai
```

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool, ScrapeWebsiteTool
```

```
=====
```

步骤1：定义工具

```
=====
```

```
searchtool = SerperDevTool() # Google搜索工具
scrapetool = ScrapeWebsiteTool() # 网页抓取工具
```

```
=====
```

步骤2：定义Agent角色

```
=====
```

研究员Agent

```
researcher = Agent(
    role="资深市场研究员",
    goal="发现关于{topic}的突破性技术和市场趋势",
    backstory=(
        "你是一位经验丰富的市场研究员，擅长从海量信息中提取关键洞察。"
        "你对AI行业有深刻理解，能够识别真正有潜力的技术和公司。"
    ),
    verbose=True,
    allow_delegation=False, # 不允许将任务委托给其他Agent
    tools=[searchtool, scrapetool]
)
```

撰写者Agent


```
w r i t e r   =   A g e n t (
    r o l e = " 技术内容总监 " ,
    g o a l = " 将研究发现转化为引人入胜、信息丰富的文章 " ,
    b a c k s t o r y = (
        " 你是一位获奖的科技作家，擅长将复杂技术概念用通俗语言解释。 "
        " 你的文章既有深度又有温度，读者总是能从你的文字中获得价值。 "
    ) ,
    v e r b o s e = T r u e ,
    a l l o w d e l e g a t i o n = F a l s e
)
```

审核者Agent

```
r e v i e w e r   =   A g e n t (
    r o l e = " 内容质量审核员 " ,
    g o a l = " 确保文章的准确性、完整性和可读性 " ,
    b a c k s t o r y = (
        " 你是一位严谨的审核员，关注事实准确性、逻辑连贯性和语言表达。 "
        " 你会指出任何不够准确或需要补充的地方。 "
    ) ,
    v e r b o s e = T r u e ,
    a l l o w d e l e g a t i o n = T r u e   #   允许委托修改任务给其他Agent
)
```

=====

步骤3：定义任务

=====

```
r e s e a r c h t a s k   =   T a s k (
    d e s c r i p t i o n = (
        " 对 { t o p i c } 进行深入研究：\ n "
        " 1 .   识别当前最重要的3个技术趋势\ n "
        " 2 .   找到每个趋势的代表性公司 / 产品\ n "
        " 3 .   分析市场规模和增长预期\ n "
        " 4 .   识别潜在的风险和挑战\ n \ n "
    )
)
```

```

" 确保所有信息都有可靠的来源。 "
),
expectedoutput = "一份包含 4 个部分的详细研究报告，每部分至少 500 字" ,
agent = researcher
)

writingtask = Task (
description = (
" 基于研究员的报告，撰写一篇关于 { topic } 的深度文章：\n "
" 1 . 标题要吸引眼球\n "
" 2 . 开头用引人入胜的 hook\n "
" 3 . 主体用具体案例支撑观点\n "
" 4 . 结尾给出前瞻性判断\n\n "
" 文章应该既专业又易读，目标读者是技术管理者。 "
),
expectedoutput = "一篇 2000 - 3000 字的专业深度文章，Markdown 格式" ,
agent = writer
)

reviewtask = Task (
description = (
" 审核文章的以下方面：\n "
" 1 . 事实准确性：所有数据和引用是否正确\n "
" 2 . 逻辑连贯性：论证是否严密\n "
" 3 . 可读性：是否通俗易懂\n "
" 4 . 完整性：是否遗漏重要信息\n\n "
" 如果发现问题，请给出具体的修改建议。 "
),
expectedoutput = "审核报告，包含评分和具体修改建议" ,
agent = reviewer
)

```

=====

步骤 4：组建 Crew 并运行

```
=====
```

```
crew = Crew(  
agents=[researcher, writer, reviewer],  
tasks=[researchtask, writingtask, reviewtask],  
process=Process.sequential, # 串行执行任务  
verbose=True  
)
```

运行

```
result = crew.kickoff(inputs={"topic": "2024年大语言模  
print(result)
```

9.3 AutoGen 详细使用教程

AutoGen 是微软开发的多Agent对话框架，核心特色是Agent之间的多轮对话来自动完成任务

```
pip install pyautogen
```

```
import autogen
```

```
=====
```

步骤1：配置LLM

```
=====
```

```
configlist = [  
{  
"model": "gpt-4",  
"apikey": "your-api-key"  
}  
]
```

```
llmconfig = {  
"configlist": configlist,  
"temperature": 0.7,  
"timeout": 120,  
}
```

```
=====
步骤2：创建Agent
=====
```

用户代理（代表人类用户）

```
userproxy = autogen.UserProxyAgent(
    name="User",
    systemmessage="你是一个用户，提出需求并审核结果。",
    codeexecutionconfig={
        "workdir": "coding", # 代码执行的工作目录
        "usedocker": False, # 是否在Docker中执行
    },
    humaninputmode="NEVER", # NEVER / ALWAYS / LASTN
)
```

助手Agent

```
assistant = autogen.AssistantAgent(
    name="Assistant",
    systemmessage=(
        "你是一个编程助手。编写代码解决问题时，请：\n"
        "1. 先分析需求\n"
        "2. 编写完整可运行的代码\n"
        "3. 包含测试用例\n"
        "4. 代码中使用中文注释"
    ),
    llmconfig=llmconfig,
)
```

代码审查Agent

```
reviewer = autogen.AssistantAgent(
    name="CodeReviewer",
    systemmessage=(
        "你是一个严格的代码审查员。检查代码的：\n"
        "1. 正确性：逻辑是否正确\n"
    )
)
```

```
" 2 . 安全性：是否有安全漏洞 \ n "
" 3 . 性能：是否可以优化 \ n "
" 4 . 风格：是否符合 P y t h o n 编码规范 \ n \ n "
" 如果发现问题，请给出具体修改建议。 "
```

```
),
llmconfig = llmconfig,
)
```

=====

步骤3：创建群聊

=====

```
groupchat = autogen.GroupChat(
agents=[userproxy, assistant, reviewer],
messages=[],
maxround=10, # 最大对话轮次
)
```

```
manager = autogen.GroupChatManager(
groupchat=groupchat,
llmconfig=llmconfig,
)
```

=====

步骤4：发起对话

=====

```
userproxy.initiatechat(
manager,
message="请编写一个 P y t h o n 函数，实现 L R U 缓存，要求：1 ) 支持 g e t 和 p u t 操作  
线程安全"
)
```

9.4 LangGraph 详细使用教程

LangGraph 是 LangChain 生态中的 workflow 编排框架，用有向图定义 Agent 的工作流，支持循环、条件分支和状态管理。

```

pip install langgraph langchain-openai

from typing import TypedDict, Annotated, Literal
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode
from langchainopenai import ChatOpenAI
from langchaincore.messages import HumanMessage,

```

```

=====

```

步骤1：定义状态

```

=====

```

```

class AgentState(TypedDict):
    messages: list # 对话历史
    next_agent: str # 下一个要执行的Agent
    task_complete: bool # 任务是否完成

```

```

=====

```

步骤2：定义Agent节点

```

=====

```

```

def researchernode(state: AgentState) -> dict:
    """研究员节点"""
    messages = state["messages"]

    llm = ChatOpenAI(model="gpt-4")
    response = llm.invoke([
        SystemMessage(content="你是研究员，负责搜集和分析信息。"),
        messages
    ])

    return {
        "messages": messages + [response],
        "next_agent": "writer" # 路由到撰写者
    }

```

```

def writernode(state: AgentState) -> dict:
    """撰写者节点"""
    messages = state["messages"]

    llm = ChatOpenAI(model="gpt-4")
    response = llm.invoke([
        SystemMessage(content="你是撰写者，将研究内容写成文章。"),
        messages
    ])

    return {
        "messages": messages + [response],
        "nextagent": "reviewer" # 路由到审核者
    }

def reviewernode(state: AgentState) -> dict:
    """审核者节点"""
    messages = state["messages"]

    llm = ChatOpenAI(model="gpt-4")
    response = llm.invoke([
        SystemMessage(content=(
            "你是审核者。如果内容质量满意，回复 'APPROVED'。"
            "否则回复 'REVISIONNEEDED' 并说明需要修改的地方。"
        )),
        messages
    ])

    # 根据审核结果决定路由
    if "APPROVED" in response.content:
        return {
            "messages": messages + [response],
            "taskcomplete": True,
            "nextagent": "end"
        }

```

```

else:
    return {
        "messages": messages + [response],
        "taskcomplete": False,
        "nextagent": "writer" # 回到撰写者修改
    }

```

=====

步骤3：定义路由函数

=====

```

def routeagent(state: AgentState) -> str:
    """根据状态决定下一个节点"""
    if state.get("taskcomplete", False):
        return "end"
    return state.get("nextagent", "researcher")

```

=====

步骤4：构建工作流图

=====

```

workflow = StateGraph(AgentState)

```

添加节点

```

workflow.addnode("researcher", researchernode)
workflow.addnode("writer", writernode)
workflow.addnode("reviewer", reviewernode)

```

添加边（定义工作流）

```

workflow.addedge("researcher", "writer")
workflow.addconditionaledges(
    "reviewer",
    routeagent,
    {
        "writer": "writer",

```



```

"end": END
}
)
workflow.addedge("writer", "reviewer")

```

设置入口

```

workflow.setentrypoint("researcher")

```

编译图

```

app = workflow.compile()

```

=====

步骤5：运行 workflow

=====

```

initialstate = {
  "messages": [HumanMessage(content="请研究并撰写一篇关于RAG技术")],
  "nextagent": "researcher",
  "taskcomplete": False
}

```

```

result = app.invoke(initialstate)
print(result["messages"][-1].content)

```

9.5 多Agent通信协议

多Agent系统中，Agent之间的通信方式直接影响系统效率：

通信方式 说明 优点 缺点

直接消息 Agent直接发送消息给指定Agent 简单直接 需要知道对方身份

共享黑板 Agent读写共享状态空间 解耦，灵活 一致性问题

发布-订阅 Agent订阅感兴趣的事件 松耦合 消息延迟

中间人 通过管理Agent中转消息 可控，可监控 单点瓶颈

9.6 角色分配和协作模式最佳实践

角色分配原则：

1. 每个 Agent 有明确的角色定义和职责边界
2. Agent 之间通过接口（而非内部状态）通信
3. 避免角色重叠导致的冗余和冲突
4. 设置一个协调者 Agent 处理冲突和优先级

协作模式选择：

- 串行：适合有严格依赖关系的任务（研究→写作→审核）
- 并行：适合独立的子任务（同时搜索多个数据源）
- 层级：适合大型复杂任务（项目经理→组长→组员）
- 辩论：适合需要多角度思考的决策（正方 vs 反方 vs 裁判）

- - -

十、Agent 评估方法

10.1 评估维度

维度 说明 评估方法

任务完成率 Agent 是否能完成指定任务 在测试集上的成功率

步骤效率 完成任务用了多少步 平均步数（越少越好）

工具使用准确率 是否选择了正确的工具和参数 工具调用准确率

成本效率 完成任务消耗了多少Token Token数 / API调用次数

鲁棒性 面对异常输入的表现 边界情况测试

延迟 从输入到输出的时间 端到端延迟

10.2 评估框架

```
class AgentEvaluator:
```

```
    """ Agent 评估框架 """
```

```
    def init(self):
```

```
        self.results = []
```

```
    def evaluate_task(self, agent, task: str, expected_output: str):
```

```
        """ 评估单个任务 """
```

```

import time

starttime = time.time()
stepstaken = 0
toolscalled = []
tokensused = 0

# 运行Agent并记录过程
result = agent.run(task)
endtime = time.time()

# 计算指标
evaluation = {
    "task": task,
    "success": self.checksuccess(result, expectedoutput),
    "timetaken": endtime - starttime,
    "stepstaken": stepstaken,
    "toolscalled": toolscalled,
    "tokensused": tokensused
}

self.results.append(evaluation)
return evaluation

def checksuccess(self, result: str, expected: str) -> bool:
    """ 检查结果是否符合预期（可以使用LLM来评估） """
    # 简化处理：检查关键词
    return expected.lower() in result.lower()

def getsummary(self) -> dict:
    """ 获取评估汇总 """
    if not self.results:
        return {}

    total = len(self.results)
    success = sum(1 for r in self.results if r["success"])

```

```

return {
    "totaltasks": total,
    "successrate": success / total,
    "avgtime": sum(r["timetaken"] for r in self.results) / len(self.results),
    "avgsteps": sum(r["stepstaken"] for r in self.results) / len(self.results)
}

```

10.3 常用评估基准

基准 说明 任务类型

WebArena 真实Web环境中的Agent评估 Web操作

SWE-bench 软件工程任务评估 代码修复

HumanEval 代码生成评估 编程

AgentBench 多场景综合评估 多种

ToolBench 工具调用能力评估 API调用

- - -

十一、Agent 的安全性问题

11.1 主要安全风险

风险 说明 防御措施

Prompt注入 恶意输入劫持Agent行为 输入验证、指令隔离

权限越界 Agent执行了超出授权的操作 最小权限原则

数据泄露 Agent将敏感信息发送给第三方 数据脱敏、审计日志

无限循环 Agent陷入死循环消耗资源 最大迭代限制

工具滥用 Agent调用危险工具 工具白名单、人工确认

成本失控 Agent过度调用API 预算限制、用量监控

11.2 安全防护实现

```

class SafeAgentWrapper:

```

```

    """安全的Agent包装器"""

```

```

def init(self, agent, maxiterations=10, maxcost=1,
dangeroustools=None, sensitivedatapatterns=None):
    self.agent = agent
    self.maxiterations = maxiterations
    self.maxcost = maxcost # 最大美元花费
    self.currentcost = 0.0
    self.dangeroustools = dangeroustools or ["delete"]
    self.sensitivedatapatterns = sensitivedatapatterns or [
        r'\b\d{16}\b', # 信用卡号
        r'\b\d{17}[\dXx]\b', # 身份证号
    ]
    self.auditlog = []

def validateinput(self, userinput: str) -> bool:
    """验证用户输入是否安全"""
    # 检测潜在的Prompt注入
    injectionpatterns = [
        "ignore previous instructions",
        "forget your role",
        "you are now",
        "system prompt",
    ]
    lowerinput = userinput.lower()
    for pattern in injectionpatterns:
        if pattern in lowerinput:
            self.auditlog.append(f"⚠ 检测到潜在注入: {pattern}")
            return False
    return True

def validatetoolcall(self, toolname: str, arguments: dict) -> bool:
    """验证工具调用是否安全"""
    # 检查危险工具
    if toolname in self.dangeroustools:

```

```

self.auditlog.append(f"        拦截危险工具调用： {toolname}")
return False

# 检查参数中的敏感数据
import re
argsstr = str(arguments)
for pattern in self.sensitivedatapatterns:
    if re.search(pattern, argsstr):
        self.auditlog.append(f"⚠ 检测到敏感数据： {pattern}")
        return False

return True

def run(self, userinput: str) -> str:
    """安全运行Agent"""
    # 输入验证
    if not self.validateinput(userinput):
        return "输入检测到潜在风险，已被拦截。"

    # 运行Agent（需要在Agent内部加入工具调用验证）
    result = self.agent.run(userinput)

    return result

- - -

```

十二、Agent 的典型应用场景详解

12.1 代码助手 Agent

```

class CodeAssistantAgent:
    """代码助手 Agent"""

    def init(self):
        self.tools = {
            'readfile': self.readfile,
            'writefile': self.writefile,

```

```

'runcode': self.runcode,
'searchdocs': self.searchdocs,
'runtests': self.runtests,
}

def readfile(self, path: str) -> str:
    """读取文件内容"""
    try:
        with open(path, 'r') as f:
            return f.read()
    except FileNotFoundError:
        return f"文件 {path} 不存在"

def writefile(self, path: str, content: str) -> str:
    """写入文件"""
    with open(path, 'w') as f:
        f.write(content)
    return f"已写入 {path}"

def runcode(self, code: str) -> str:
    """执行代码"""
    import subprocess
    result = subprocess.run(
        ['python', '-c', code],
        captureoutput=True, text=True, timeout=30
    )
    return result.stdout or result.stderr

def searchdocs(self, query: str) -> str:
    """搜索技术文档"""
    # 实际实现中调用搜索API
    return f"关于 '{query}' 的文档搜索结果..."

def runtests(self, testpath: str) -> str:
    """运行测试"""

```

```

import subprocess
result = subprocess.run(
    ['python', '-m', 'pytest', testpath, '-v'],
    captureoutput=True, text=True, timeout=60
)
return result.stdout

def run(self, task: str) -> str:
    """运行代码助手"""
    # 1. 理解任务
    # 2. 读取相关代码
    # 3. 编写 / 修改代码
    # 4. 运行测试验证
    # 5. 返回结果
    pass # 实际实现使用LLM驱动

```

12.2 研究助手 Agent

```

class ResearchAssistantAgent:
    """研究助手 Agent"""

    def init(self):
        self.tools = {
            'searchpapers': self.searchpapers,
            'summarizepaper': self.summarizepaper,
            'comparemethods': self.comparemethods,
            'generatereport': self.generatereport,
        }

    def searchpapers(self, query: str, yearfrom: int):
        """搜索学术论文"""
        # 调用 Semantic Scholar API 或 ArXiv API
        return [
            {"title": "Attention Is All You Need", "year": 2017},
            {"title": "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", "year": 2019},
        ]

```



```

" citations": 80000},
]

def summarizepaper(self, paperid: str) -> str:
    """总结论文"""
    return "论文总结..."

def comparemethods(self, methoda: str, methodb: str) -> str:
    """比较不同方法"""
    return f"{methoda} vs {methodb} 的对比分析..."

def generatereport(self, topic: str, findings: list) -> str:
    """生成研究报告"""
    return f"# {topic} 研究报告\n\n## 主要发现\n..."

```

12.3 数据分析 Agent

```

class DataAnalysisAgent:
    """数据分析 Agent"""

    def __init__(self):
        self.tools = {
            'loaddata': self.loaddata,
            'describedata': self.describedata,
            'visualize': self.visualize,
            'runstatisticaltest': self.runstatisticaltest,
            'buildmodel': self.buildmodel,
        }

    def loaddata(self, path: str) -> str:
        """加载数据"""
        import pandas as pd
        df = pd.readcsv(path)
        return f"数据形状: {df.shape}\n列名: {list(df.columns)}"

    def describedata(self, dataid: str) -> str:

```

```

""" 描述性统计 """
return "描述性统计结果..."

def visualize(self, dataid: str, charttype: str,
""" 生成可视化 """
import matplotlib.pyplot as plt
# 生成图表并保存
return "图表已生成: visualization.png"

def runstatisticaltest(self, dataid: str, testtype: str,
""" 运行统计检验 """
from scipy import stats
return "统计检验结果..."

def buildmodel(self, dataid: str, target: str, modeltype: str,
""" 构建预测模型 """
return "模型训练结果..."

```

12.4 客服 Agent

```

class CustomerServiceAgent:
""" 客服 Agent """

def __init__(self):
self.tools = {
'searchknowledgebase': self.searchknowledgebase,
'checkorder': self.checkorder,
'createticket': self.createticket,
'escalate': self.escalatetohuman,
}
self.maxresolveattempts = 3

def searchknowledgebase(self, query: str) -> str:
""" 搜索知识库 """
return f"关于 '{query}' 的常见解答..."

```

```

def checkorder(self, orderid: str) -> dict:
    """ 查询订单状态 """
    return {"orderid": orderid, "status": "shipping",

def createticket(self, issue: str, priority: str):
    """ 创建工单 """
    return f"工单已创建: TK-{hash(issue) % 10000:04d}"

def escalatetohuman(self, reason: str) -> str:
    """ 转人工客服 """
    return "已转接人工客服,请稍等..."

def run(self, usermessage: str) -> str:
    """ 处理客服对话 """
    # 1. 理解用户问题
    # 2. 搜索知识库
    # 3. 尝试解决
    # 4. 如果无法解决,创建工单或转人工
    pass
- - -

```

十三、Agent 框架对比

维度	LangChain	LangGraph	CrewAI	AutoGen	MetaGPT
----	-----------	-----------	--------	---------	---------

核心理念	灵活的工作流编排	角色驱动的团队协作	多Agent对话	软件开发流程模拟	
------	----------	-----------	----------	----------	--

学习曲线	中等	低	中等	高	
------	----	---	----	---	--

灵活性	高(图结构)	中等	高	低(固定流程)	
-----	--------	----	---	---------	--

适用场景	通用	团队协作	多Agent交互	软件开发	
------	----	------	----------	------	--

状态管理	LangGraph优秀	内置	内置	内置	
------	-------------	----	----	----	--

工具集成	丰富	中等	中等	有限	
------	----	----	----	----	--

社区活跃度	高	中等	高	中等	
-------	---	----	---	----	--

生产就绪度	中等	低	低	低	
-------	----	---	---	---	--

选择建议

- 快速原型：CrewAI（简单直觉）
 - 复杂工作流：LangGraph（图结构灵活）
 - 研究 / 对话：AutoGen（对话式交互）
 - 软件开发：MetaGPT（流程标准化）
 - 生产部署*：LangGraph + 自定义安全层
- - -

十四、从零构建一个完整 Agent 的详细教程

下面我们从零开始，不依赖任何框架，构建一个功能完整的 Agent。

"""

从零构建一个完整的 Agent

功能：能够搜索信息、执行代码、管理记忆、自我纠错

"""

```
import json
import re
import time
from typing import List, Dict, Optional, Callable
from openai import OpenAI
```

```
client = OpenAI(apikey="your-api-key")
```

```
=====
```

组件1：记忆系统

```
=====
```

```
class MemorySystem:
```

```
    """ Agent 记忆系统 """
```

```
    def init(self, maxshortterm=10):
```

```
        self.shortterm = [] # 短期记忆（最近对话）
```

```
        self.longterm = [] # 长期记忆（重要事实）
```

```
        self.working = {} # 工作记忆（当前任务状态）
```

```
        self.maxshortterm = maxshortterm
```

```

def addshortterm(self, role: str, content: str):
    """添加短期记忆"""
    self.shortterm.append({"role": role, "content": content})
    # 超出限制时压缩
    if len(self.shortterm) > self.maxshortterm:
        self.compressshortterm()

def addlongterm(self, content: str, importance: float):
    """添加长期记忆"""
    self.longterm.append({"content": content, "importance": importance})

def setworking(self, key: str, value):
    """设置工作记忆"""
    self.working[key] = value

def getworking(self, key: str, default=None):
    """获取工作记忆"""
    return self.working.get(key, default)

def getcontext(self, query: str = None) -> str:
    """获取当前记忆上下文"""
    parts = []

    if self.working:
        parts.append("【工作记忆】" + json.dumps(self.working, ensure_ascii=False))

    if self.shortterm:
        recent = self.shortterm[-6:]
        parts.append("【近期对话】\n" + "\n".join(
            f" {m['role']}: {m['content'][:100]}..." for m in recent
        ))

    if self.longterm and query:
        # 简单关键词匹配检索
        relevant = [m for m in self.longterm if any(kw in m['content'] for kw in query.split())]

```

```

if relevant:
    parts.append("【相关记忆】\n" + "\n".join(
        f" [{m['importance']:.1f}] {m['content']}" for m
    ))

return "\n\n".join(parts)

def compressshortterm(self):
    """压缩短期记忆"""
    # 保留最近的一半，其余尝试提取关键信息转长期记忆
    half = len(self.shortterm) // 2
    oldmessages = self.shortterm[:half]
    self.shortterm = self.shortterm[half:]

    # 将旧消息中的重要信息提取到长期记忆
    for msg in oldmessages:
        if len(msg["content"]) > 50: # 较长的消息可能包含重要信息
            self.addlongterm(msg["content"][:200], importance

```

组件2：工具系统

```

=====

class ToolSystem:
    """Agent 工具系统"""

    def __init__(self):
        self.tools = {} # 工具名 -> 工具定义
        self.functions = {} # 工具名 -> 执行函数

    def register(self, name: str, description: str, p
        """注册一个工具"""

        self.tools[name] = {
            "type": "function",
            "function": {
                "name": name,

```

```

    "description": description,
    "parameters": parameters
}
}

self.functions[name] = func

def gettooldefinitions(self) -> list:
    """获取OpenAI格式的工具定义列表"""
    return list(self.tools.values())

def execute(self, name: str, arguments: dict) ->
    """执行工具"""
    if name not in self.functions:
        return json.dumps({"error": f"工具 {name} 不存在"})

    try:
        result = self.functions[name]
        if isinstance(result, str):
            return result
        return json.dumps(result, ensure_ascii=False)
    except Exception as e:
        return json.dumps({"error": str(e)})

```

=====

组件3：规划系统

=====

```

class PlanningSystem:
    """Agent 规划系统"""

    def init(self, client, model="gpt-4"):
        self.client = client
        self.model = model

    def createplan(self, task: str, availabletools: list):
        """为任务创建执行计划"""

```

```

toolnames = [t["function"]["name"] for t in available_tools]

response = self.client.chat.completions.create(
    model=self.model,
    messages=[
        {
            "role": "system",
            "content": (
                "你是一个任务规划专家。给定任务和可用工具列表，制定执行计划。 \n"
                "计划应该是一个步骤列表，每个步骤说明： \n"
                "1. 要做什么 \n"
                "2. 使用哪个工具 \n"
                "3. 预期输出 \n \n"
                f"可用工具：{', '.join(toolnames)} \n"
                "以JSON列表格式输出。"
            )
        },
        {"role": "user", "content": f"任务：{task}" }
    ],
    response_format={"type": "jsonobject"}
)

result = json.loads(response.choices[0].message.content)
return result.get("steps", [])

```

=====

组合：完整Agent

=====

```

class CompleteAgent:
    """从零构建的完整 Agent"""

    def __init__(self, model="gpt-4", maxiterations=10):
        self.model = model
        self.maxiterations = maxiterations

```



```
# 初始化三大系统
```

```
self.memory = MemorySystem()
```

```
self.tools = ToolSystem()
```

```
self.planner = PlanningSystem(client, model)
```

```
# 注册内置工具
```

```
self.registerdefaulttools()
```

```
def registerdefaulttools(self):
```

```
    """注册默认工具集"""
```

```
    self.tools.register(
```

```
        name="calculate",
```

```
        description="执行数学计算",
```

```
        parameters={
```

```
            "type": "object",
```

```
            "properties": {
```

```
                "expression": {"type": "string", "description": "数学表达式"},
```

```
            "required": ["expression"]
```

```
        },
```

```
        func=lambda expression: str(eval(expression))
```

```
    self.tools.register(
```

```
        name="search",
```

```
        description="搜索互联网获取信息",
```

```
        parameters={
```

```
            "type": "object",
```

```
            "properties": {
```

```
                "query": {"type": "string", "description": "搜索关键词"},
```

```
            "required": ["query"]
```

```
        },
```

```
        func=lambda query: f"搜索'{query}'的结果：相关内容..."
```

)

```
self.tools.register(  
    name="save_note",  
    description="保存重要信息到长期记忆",  
    parameters={  
        "type": "object",  
        "properties": {  
            "content": {"type": "string", "description": "要保存的内容"},  
            "importance": {"type": "number", "description": "重要性 (0.0 到 1.0)"},  
        },  
        "required": ["content"]  
    },  
    func=lambda content, importance=0.8: self.save_to_memory(content, importance)  
)
```

```
def save_to_memory(self, content: str, importance: float):  
    """保存到记忆"""  
    self.memory.add_longterm(content, importance)  
    return f"已保存到长期记忆 (重要性: {importance})"
```

```
def run(self, user_input: str) -> str:  
    """运行Agent"""  
    print(f"\n{'='*60}")  
    print(f"      用户: {user_input}")  
    print(f"{'='*60}\n")
```

记录用户输入

```
self.memory.add_shortterm("user", user_input)
```

构建消息列表

```
messages = self.build_messages(user_input)
```

ReAct 循环

```
for i in range(self.max_iterations):
```

```

print ( f " - - - 迭代 { i + 1 } - - - " )

# 调用LLM
response = client.chat.completions.create(
model=self.model,
messages=messages,
tools=self.tools.gettooldefinitions(),
toolchoice="auto"
)

message = response.choices[0].message
messages.append(message)

# 处理工具调用
if message.toolcalls:
for toolcall in message.toolcalls:
funcname = toolcall.function.name
funcargs = json.loads(toolcall.function.arguments)

print ( f "          调用: { funcname } ( { funcargs } ) " )

# 执行工具
result = self.tools.execute(funcname, funcargs)
print ( f "          结果: { result[:100] } . . . " )

# 添加到消息
messages.append( {
"role": "tool",
"toolcallid": toolcall.id,
"content": result
} )

# 记录到记忆
self.memory.addshortterm("toolresult", f"{funcname}
else:
# LLM直接回复

```

```

if message.content:
    print(f"\n    Agent: {message.content[:200]}...")
    self.memory.addshortterm("assistant", message.content)
    return message.content

return "达到最大迭代次数，任务未完成。"

def buildmessages(self, userinput: str) -> list:
    """ 构建发送给LLM的消息列表 """
    # 获取记忆上下文
    memorycontext = self.memory.getcontext(userinput)

    messages = [
        {
            "role": "system",
            "content": (
                "你是一个智能助手，能够通过思考和工具调用来帮助用户。 \n "
                "请使用 ReAct 模式工作：先思考 ( Thought )，再行动 ( Action )，然后观察 ( Observation )。 \n "
                f"当前记忆上下文： \n {memorycontext} "
            )
        },
        {"role": "user", "content": userinput}
    ]

    return messages

=====
运行Agent
=====
agent = CompleteAgent()
result = agent.run("帮我计算 (15 + 27) * 3 的结果，并保存这个计算过程")

- - -

```

十五、推荐学习资源

资源 链接 说明

Google 5天智能体课程 <https://www.kaggle.com/learn-guide>

微软 AI Agents 入门课程 <https://github.com/microsoft/ai>

Hello-Agents (Datawhale) <https://github.com/datawhale>

HuggingFace 智能体课程 <https://github.com/huggingface/>

LangGraph 官方文档 <https://langchain-ai.github.io/lang>

CrewAI 官方文档 <https://docs.crewai.com/> 角色驱动

AutoGen 官方文档 <https://microsoft.github.io/autogen/>

MCP 官方文档 <https://modelcontextprotocol.io/> 工具协议

- - -

核心总结：Agent = LLM + 规划 + 工具 +

记忆。Agent的核心突破在于从"生成文本"到"生成行动并执行行动"的跃迁，让AI从被动应答变为主

Calling是最核心的工具调用机制，MCP正在成为工具协议的统一标准，而记忆系统则是Agent持